

Systems programming

Lab 5 - SDL2 graphical applications

Learning objectives

Graphical applications development

Event based programming

In this laboratory students will start using the SDL2 graphical library to implement graphical applications.

SDL2 is a library that allows the creation of graphical applications:

- create graphical window
- draw geometric shapes
- draw text in various fonts styles and sizes
- read keyboard events
- read mouse events

1 SDL2 installation

To use this library it is necessary to install it on Linux or macOS. By installing additional plugins to SDL it is possible to enhance the basic functionalities.

<https://wiki.libsdl.org/SDL2/Installation>

<https://wiki.libsdl.org/SDL2/Libraries>

1.1 Linux

The **SDL2** installation in Linux is straightforward and depends on the Linux version.

For Ubuntu it is necessary to issue the following commands

```
sudo apt install libsdl2-dev
sudo apt install libsdl2-image-dev
sudo apt install libsdl2-ttf-dev
sudo apt install libsdl2-gfx-dev
```

For other distributions, the packages to install are different, as explained in the following page:

1.2 MAC OS X

To install the SDL2 library in MAC OS X it is necessary to use the **brew** package manager:

<https://brew.sh/>

after installing brew it is necessary to type the following command in the terminal:

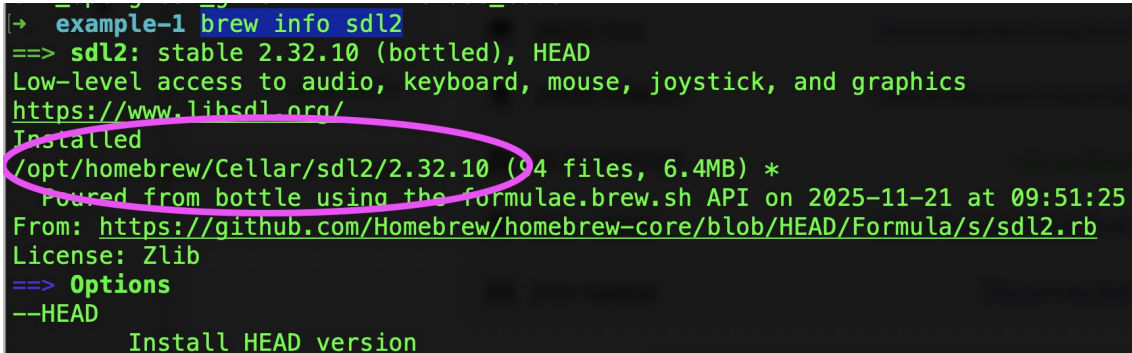
```
brew install sdl2
brew install sdl2_image
brew install sdl2_ttf
brew install sdl2_gfx
```

MAC OS X

In mac os X, brew installs libraries in non default directories, and as such gcc does not automatically finds the .h files and the libraries.

1 - Verify where the zeromq files were installed running the command

brew info sdl2



```
example-1 brew info sdl2
==> sdl2: stable 2.32.10 (bottled), HEAD
Low-level access to audio, keyboard, mouse, joystick, and graphics
https://www.libsdl.org/
Installed
/opt/homebrew/Cellar/sdl2/2.32.10 (94 files, 6.4MB) *
  Poured from bottle using the formulae.brew.sh API on 2025-11-21 at 09:51:25
From: https://github.com/Homebrew/homebrew-core/blob/HEAD/Formula/s/sdl2.rb
License: Zlib
==> Options
--HEAD
      Install HEAD version
```

2 – Use the installation directory when compiling your application and in the makefile:

```
gcc -g example-1.c -o example-1 -lSDL2 \
    -I/opt/homebrew/Cellar/sdl2/2.32.10/include \
    -L/opt/homebrew/Cellar/sdl2/2.32.10/lib
```

Unfortunately other brew packages were installed in different directories.....:

brew info sdl2_gfx

brew info sdl2_image

brew info sdl2_ttf

If the package was installed in a different directory the command **brew info sdl2** will print that location.

To help Mac OS X students we provide different makefiles to be used there.

To compile with all the libraries we get a makefile like this:

```
gcc -g example.c -o example \  
    -lSDL2 -lSDL2_image -lSDL2_gfx -lSDL2_ttf\  
    -I/opt/homebrew/Cellar/sdl2/2.32.10/include \  
    -L/opt/homebrew/Cellar/sdl2/2.32.10/lib \  
    -I/opt/homebrew/Cellar/sdl2/2.32.10/include/SDL2 \  
    -L/opt/homebrew/Cellar/sdl2_image/2.8.8/lib \  
    -I/opt/homebrew/Cellar/sdl2_image/2.8.8/include \  
    -I/opt/homebrew/Cellar/sdl2_ttf/2.24.0/include \  
    -L/opt/homebrew/Cellar/sdl2_ttf/2.24.0/lib \  
    -I/opt/homebrew/Cellar/sdl2_gfx/1.0.4/include \  
    -L/opt/homebrew/Cellar/sdl2_gfx/1.0.4/lib \
```

2 SDL2 tutorial and documentation

The following site presents a detailed documentation of the various functions provided by SDL2:

<https://wiki.libsdl.org/SDL2/FrontPage>

<https://wiki.libsdl.org/SDL2/Introduction>

<https://wiki.libsdl.org/SDL2/APIByCategory>

Existing tutorials may be of help to further understand how to program graphical applications using SDL2:

<https://wiki.libsdl.org/SDL2/Tutorials>

<https://glusoft.com/sdl2-tutorials/>

<https://lazyfoo.net/tutorials/SDL/index.php>

<https://wiki.libsdl.org/SDL2/APIByCategory>

3 Examples

The provided example files demonstrate the use of the fundamental SDL2 functions.

3.1 Example1 – Graphical windows

<https://wiki.libsdl.org/SDL2/CategoryInit>
<https://wiki.libsdl.org/SDL2/CategoryVideo>
<https://wiki.libsdl.org/SDL2/CategoryRender>

example-1.c The example-1.c application opens a graphical windows and cycles its background over all the RGB colors. IN every iteration this application updates a RGB color, clears the object/renderer that represents the windows contents and updates the graphical window.

To compile this examples only the **-ISDL2** linking option is necessary

<pre>SDL_Init(SDL_INIT_EVERYTHING)</pre>	Initialization of the SDL library
<pre>SDL_Window* win = SDL_CreateWindow("EXAMPLE 1", SDL_WINDOWPOS_CENTERED, SDL_WINDOWPOS_CENTERED, 1000, 1000, 0);</pre>	Creation of a 1000x1000 pixel window, with "EXAMPLE 1" title
<pre>Uint32 render_flags = SDL_RENDERER_ACCELERATED; SDL_Renderer* rend = SDL_CreateRenderer(win, -1, render_flags);</pre>	Creates an object/rendered that will represent the windows content and tries to user HW graphical acceleration hardware. All graphical operations (drawings) will be done on this object and copied to the screen when suitable.
<pre>SDL_Event event; SDL_PollEvent(&event); if (event.type== SDL_QUIT){ break; }</pre>	Verifies if the user closed the window.
<pre>SDL_Color backgroud_color; backgroud_color.r = 255; backgroud_color.g = 0; backgroud_color.b = 255; backgroud_color.a = 255;</pre>	Definitions of the background color in SDL_Color structure. This variable will be updated on every loop iteration.

<pre>SDL_SetRenderDrawColor(rend, backgroud_color.r, backgroud_color.g, backgroud_color.b, backgroud_color.a); SDL_RenderClear(rend);</pre>	Update of the color that the renderer will use in the next operation. Clear the rendered with the previously updated color
<pre>SDL_RenderPresent(rend);</pre>	Update of the screen with the contents of the renderer.

It is better to run this program inside VSCode debugger because it does not contain any way to orderly close the window and terminate the application.

3.2 Example 2 – Geometric figures

https://www.ferzkopp.net/Software/SDL2_gfx/Docs/html/index.html

example-2.c The example-2.c application opens a graphical windows and randomly draw various graphical objects:

- random object (circle, ellipse, rectangle)
- random color
- random size

To compile this examples the **-ISDL2 -ISDL2_gfx** linking option are necessary.

<pre>while(1) { . . . SDL_SetRenderDrawColor(rend, backgroud_color.r, backgroud_color.g, backgroud_color.b, backgroud_color.a); SDL_RenderClear(rend); ...</pre>	In the beginning of each iteration, the screen is cleaned. In this application the window may not close properly :(
<pre>filledCircleColor(rend, x, y, height, SDL_ColorToUint(object_color)); circleColor(rend, x, y, height, SDL_ColorToUint(object_color));</pre>	Draws a circle/ circumference into the renderer with the color in object_color.

<pre>boxColor(rend, x, y, x+width, y+height, SDL_ColorToUint(object_color)); rectangleColor(rend, x, y, x+width, y+height, SDL_ColorToUint(object_color));</pre>	Draws a filled/empty rectangle into the renderer with the color in object_color
<pre>ellipseColor(rend, x, y, width, height, SDL_ColorToUint(object_color)); filledEllipseColor(rend, x, y, width, height, SDL_ColorToUint(object_color));</pre>	Draws a filled/empty ellipse into the renderer with the color in object_color
<pre>SDL_RenderPresent(rend); SDL_Delay(50); }</pre>	In the end of each loop iteration the rendered contents are transferred to the screen/window. If the rendered was cleared, only one object is presented, otherwise various objects appear.

It is better to run this program inside VSCode debugger because it does not contain any way to orderly close the window and terminate the application.

3.3 Example 3 – Writing text

https://wiki.libsdl.org/SDL2_ttf/CategoryAPI

example-3.c The example-3.c application opens a graphical windows and randomly draw a string. The string is draw rotated in various angles. Before each string is draw the windows is cleared.

To compile this examples the **-ISDL2 -ISDL2_ttf** linking option are necessary.

<pre>TTF_Init(); TTF_Font* font = TTF_OpenFont("./SuperChiby.ttf", 100);</pre>	Initialization of the font manager. Load of a certain font file is the determined font size. The .ttf font file should be present in the directory of the application.
<pre>while(1) { SDL_Event event; while (SDL_PollEvent(&event)) {} SDL_SetRenderDrawColor(rend, backgroud_color.r, backgroud_color.g,</pre>	In the beginning of each iteration, the system handles pending events and clears the screen.

<pre> backgroud_color.b, backgroud_color.a); SDL_RenderClear(rend); ... </pre>	
<pre> SDL_Surface* textSurface = TTF_RenderText_Solid(font, "HELLO SDL_TTF", object_color); SDL_Texture* textTexture = SDL_CreateTextureFromSurface(rend, textSurface); SDL_FreeSurface(textSurface); </pre>	Creation of a Surface object with the rendering of the text using the opened font and with a certain color. Transformation of the surface object into a Texture Free of the temporary surface
<pre> SDL_Rect textDest = { x, y, 1000, 0 }; SDL_QueryTexture(textTexture, NULL, NULL, &textDest.w, &textDest.h); </pre>	Definition of the future position of the text Extraction of the width and height of the rendered text.
<pre> for (angle =0; angle <180; angle +=45) { SDL_RenderCopyEx(rend, textTexture, NULL, &textDest, angle, NULL, SDL_FLIP_NONE); } SDL_DestroyTexture(textTexture); </pre>	Drawing of the text with size/location in textDest and rotated by angle Destruction of th texture
<pre> SDL_RenderPresent(rend); SDL_Delay(50); } </pre>	In the end of each loop iteration the rendered contents are transferred to the screen/window.

It is better to run this program inside VSCode debugger because it does not contain any way to orderly close the window and terminate the application.

3.4 Example 4 – Drawing images and mouse events

<https://wiki.libsdl.org/SDL2/CategoryEvents>

https://wiki.libsdl.org/SDL2/SDL_EventType

https://wiki.libsdl.org/SDL2/SDL_MouseMotionEvent

example-4.c The example-4.c application opens a graphical windows and allows the user to draw the symbol os IST with the mouse.

Whenever the mouse is moved with the right button pressed, the IST symbol is printed.

If the user click on close button on the window top bar, the window closes.

To compile this examples the **-ISDL2 -ISDL2_image** linking option are necessary.

```
SDL_Surface* imageSurface =
    IMG_Load("ist.png");
SDL_Texture* imageTexture =
    SDL_CreateTextureFromSurface(
        rend, imageSurface);
SDL_FreeSurface(imageSurface);
```

Loads an image from disk into a surface

Loads image from the surface into the graphics hardware memory (as Texture).

Destroys temporary Surface

```
int close = 0;
while (!close) {
    SDL_Event event;

    SDL_WaitEvent(&event);
    //SDL_PollEvent(&event);
    switch (event.type) {
        ...
```

The application will be in the loop while the boolean close variable is False

Waits for an event.

Verifies what event was caught

```
case SDL_QUIT:
    close = 1;
    break;
```

User clicked the close button on the window top bar



```
case SDL_MOUSEBUTTONDOWN:
    printf("mouse down\n");
    mouse_down = 1;
    break;
case SDL_MOUSEBUTTONUP:
    printf("mouse up\n");
    mouse_down = 0;
    break;
```

The user clicked any mouse button or released it

```
case SDL_MOUSEMOTION:
    if (mouse_down) {
        SDL_Rect dst;
        dst.x = event.motion.x - 100;
        dst.y = event.motion.y - 100;
        dst.w = 200;
```

If the user moved the mouse and the **mouse_button** variable is True:

The **dst** variable will contain the location of the mouse (**event.motion**) and the size of the IST symbol (200x200)

<pre> dst.h = 200; SDL_RenderCopy(rend, imageTexture, NULL, &dst); SDL_RenderPresent(rend); } break; </pre>	Then, the imageTexture is drawn on the rendered The window is updated.
<pre> } SDL_DestroyRenderer(rend); SDL_DestroyWindow(win); SDL_Quit(); return 0; </pre>	After exiting the loop, the renderer is destroyed the window is closed/destroyed SDL library is closed

3.5 Example 5 - Reading keyboard

<https://wiki.libsdl.org/SDL2/CategoryEvents>

https://wiki.libsdl.org/SDL2/SDL_EventType

https://wiki.libsdl.org/SDL2/SDL_KeyboardEvent

https://wiki.libsdl.org/SDL2/SDL_Scancode

example-5.c In the example-5.c application the user controls (using the cursor or OPQA keys) the symbol of the IST moving it over the whole window.

To compile this examples the **-lSDL2 -lSDL2_image** linking option are necessary.

<pre> SDL_Surface* imageSurface = IMG_Load("ist.png"); SDL_Texture* imageTexture = SDL_CreateTextureFromSurface(rend, imageSurface); SDL_FreeSurface(imageSurface); </pre>	Loads an image from disk into a surface Loads image from the surface into the graphics hardware memory (as Texture). Destroys temporary Surface
<pre> SDL_Rect imageDestination; SDL_QueryTexture(imageTexture, NULL, NULL, &imageDestination.w, &imageDestination.h); imageDestination.x = 0; imageDestination.y = 0; imageDestination.w /= 3; </pre>	Extracts width and height from the IST symbol and stores it into a SDL_rect Defines initial location of the image as the left top pf the window Defines the size of the image to be draw as 1/6 of the original size

<code>imageDestination.h /= 3;</code>	
<pre>int close = 0; while (!close) { SDL_Event event; SDL_WaitEvent(&event); switch (event.type) { case SDL_QUIT: close = 1; break; ... } }</pre>	The application will be in the loop while the boolean close variable is False Waits for an event. Verifies what event was caught User clicks the close button on the window  top bar
<code>case SDL_KEYDOWN:</code>	To verify if the the user pressed any key on the keyboard, it is necessary to verify if the caught event is of type SDL_KEYDOWN
<pre>switch (event.key.keysym.scancode){ case SDL_SCANCODE_Q: case SDL_SCANCODE_UP: imageAngle = 180; imageDestination.y -= 10; break; . . . default: break; }</pre>	In this case the event variable will contain information about the pressed key. The event.key.keysym.scancode field will contain that information and can be compared with the various constants associated with each key.

3.6 Example 6 - Timer events

<https://wiki.libsdl.org/SDL2/CategoryTimer>

https://wiki.libsdl.org/SDL2/SDL_AddTimer

example-6.c In the example-6.c application the user controls a timer: by pressing the spacebar key a 5 second timer is started. Timers in SDL execute periodically a callback function.

In this example, when the timer is active, the **timer_callback** function is periodically executed and generates a **SDL_USEREVENT** is generated (**SDL_PushEvent**). The main loop handles of this event and prints a message in the terminal. In other more complex uses, the **SDL_USEREVENT** handler (in the **main**) can update the graphical window.

To compile this examples the **-ISDL2** linking option are necessary.

<code>Uint32 timer_callback(Uint32 interval,</code>	Callback function to be called by the timer
---	---

<pre> void* param){ SDL_Event timer_event; SDL_zero(timer_event); timer_event.type = SDL_USEREVENT; timer_event.user.code = 2; SDL_PushEvent(&timer_event); return interval; } </pre>	every 5 seconds (period) Initialization of a SDL_USEREVENT Push of the initialized event into the event queue. After this function returns, the main will be able to handle this event. Return of the value interval value until the next timer activation
<pre> int close = 0; while (!close) { SDL_Event event; SDL_WaitEvent(&event); switch (event.type) { </pre>	The application will be in the loop while the boolean close variable is False Waits for an event. Verifies what event was caught
<pre> case SDL_KEYDOWN: if (event.key.keysym.scancode == SDL_SCANCODE_SPACE){ if(timer_id == 0){ timer_id = SDL_AddTimer(5000, timer_callback, NULL); } else { SDL_RemoveTimer(timer_id); timer_id = 0; } } } break; </pre>	If the user presses the SPACE KEY The timer is initialized or the timer is destroyed.
<pre> case SDL_USEREVENT: if (event.user.code == 2){ printf("Timer event %d\n", n++); } break; </pre>	If it is a SDL_USEREVENT (generated by the timer callback) execute suitable code.
<pre> SDL_SetRenderDrawColor(rend, 255, 255, 255, 255); </pre>	Set the renderer background to white

<pre> SDL_RenderClear(rend); SDL_RenderCopyEx(rend, imageTexture, NULL, &imageDestination, imageAngle, NULL, SDL_FLIP_NONE); SDL_RenderPresent(rend); </pre>	Draw the IST symbol in the defined destinations (coordinates and size) and rotatin angle The window is updated.
<pre> } SDL_DestroyRenderer(rend); SDL_DestroyWindow(win); SDL_Quit(); return 0; </pre>	After exiting the loop, • the renderer is destroyed • the window is closed/destroyed • SDL library is closed

4 Exercise 1

https://wiki.libsdl.org/SDL2/SDL_WaitEvent

https://wiki.libsdl.org/SDL2/SDL_PollEvent

Run the **example_4** application at the same time as the **htop** program (on a different terminal) and observe the CPU usage of the **example_4** application when idle.

Modify the function that reads the events from **SDL_WaitEvent(&event)** to **SDL_PollEvent(&event)**.

Run again **example_4** and observe the CPU usage on **htop** when the application is idle.

What is the difference in behavior of both functions? Read the documentation page to further understand the differences.

What function should be used?

5 Exercise 2

https://wiki.libsdl.org/SDL2/SDL_MouseButtonEvent

Modify **example 4** so that only the left mouse button draws the IST symbol.

6 Exercise 3

Modify all examples so that the application can be closed by clicking in the close button on the top bar.

7 Exercise 4

https://wiki.libsdl.org/SDL2/SDL_UserEvent

https://wiki.libsdl.org/SDL2/SDL_EventType

https://wiki.libsdl.org/SDL2/SDL_Scancode

Modify **example 3** so that it continues to print the string, but now the user can control the application with the following keys:

- **q** application quits
- **+** the size of the string becomes 2 times bigger
- **-** the size of the string decreases to half

Furthermore, the application should quit when the user clicks on the close button in the application top bar.

To accomplish this follow the next steps:

- register a timer_callback that periodically sends an **SDL_USEREVENT**
- Create a main loop where the application waits for SDL events (**SDL_WaitEvent(&event)**)
- Program the event handler for the **SDL_QUIT**
- program an event handler for the **q + -** keys with the suitable behavior
- program the event handler for the periodic **SDL_USEREVENT** so that it clears the screen and draws the text.

8 Exercise 5

Modify exercise Exercise 1 from laboratory 4 so that the server shows the moving letters in a SDL2 graphical windows.